

Real-Time Posture Tracking and Visual Feedback for Improving Exercise Performance using Deep Learning

Samiul Islam Ramim¹, Md. Abdur Rahman², Fahim Sadik³, Prionti Das⁴, Md. Nazmul Abdal⁵

^{1,2,3,4,5}Department of Computer Science and Engineering, University of Liberal Arts Bangladesh, Dhaka-1207, Bangladesh

Email: samiul.islam.cse@ulab.edu.bd, abdur.rahman2.cse@ulab.edu.bd, fahim.sadik.cse@ulab.edu.bd, prionti.das.cse@ulab.edu.bd, nazmul.abdal@ulab.edu.bd

Abstract—In today's busy era, many individuals have limited access to gyms due to time or financial conditions, which leads to self-training through online resources. However, incorrect posture during exercises often results in injuries. To overcome this problem, this research proposes a deep learning-based visual feedback system for performing exercises under AI supervision. This system analyzes body landmarks using Mediapipe's model BlazePose to ensure the current posture. A classification model TCN (Temporal Convolutional Network) has been implemented to recognize the intended exercise without requiring manual input. After the exercise is detected, a regression model MLP (Multi-Layer Perceptron) gives real-time visual feedback by estimating the correct posture for that moment. The proposed TCN model is successful in detecting the exercises with an accuracy of 97.63 percent, and the MLP accomplishes an MAE (Mean Absolute Error) of 2.68 degrees to 2.86 degrees in correct posture estimation. This approach enables automatic workout recognition and correction of workouts, offering an accessible solution to self-fitness training.

Index Terms—Human Pose Estimation, AI Fitness Supervision, MediaPipe, BlazePose, TCN(Temporal Convolutional Network), MLP(Multi-Layer Perceptron), Particle Filter.

I. INTRODUCTION

We know that “Health is Wealth,” and taking care of our bodies requires regular exercise. Yet in today's fast-paced, work-heavy lifestyle, many people struggle to maintain their fitness. Financial limitations and lack of proper supervision are major obstacles. Exercising without guidance can lead to issues such as muscle pain, soreness, or shoulder, spine, and neck injuries—sometimes even requiring hospitalization [1]. Although many products offer supervised training, they tend to be expensive and mostly rely on pre-recorded or animated videos [2] [3]. Hiring a trainer or maintaining a gym membership is also costly, making proper guidance inaccessible for many.

To make self-exercising safer and more affordable, this research proposes a deep learning-based visual feedback system that works with any basic camera-equipped device. It supervises exercises by analyzing posture and showing corrective visual cues directly on the user's body. This eliminates the need for costly equipment, trainers, or subscriptions and saves valuable time.

The system captures live footage and processes it through a

classification model (TCN) that identifies the workout using MediaPipe pose keypoints. Once the posture is detected, a regression model (MLP) compares the user's joint angles with pre-trained datasets. The system then draws a ghost line alongside the user's pose, helping them adjust their form to the correct angles. A key advantage of this approach is its ability to normalize posture detection for different body types and varying distances from the camera by using 3D coordinates. We also addressed occlusion challenges by applying a particle filter, resulting in smoother and more reliable tracking.

II. RELATED WORKS

The authors of this research proposed a three-phase model combining CNN transfer learning, HPO, and image augmentation to address the challenge of training CNNs with limited data. Using this approach, they optimized AlexNet, VGG16, CNN, and MLP, achieving accuracies of 91.2% for AlexNet, 90.2% for VGG16, 87.5% for CNN, and 89.9% for MLP using the MPII dataset [4]. Another study introduced an exercise-tracking system using Mediapipe with 11 selected keypoints and an MLP model for exercise recognition, providing user feedback and achieving a 16% higher accuracy than the baseline [5]. A related work explored posture detection and exercise comparison using MediaPipe and OpenPose to improve user performance through real-time feedback supported by AI-based posture detection using live images and video sensors [6].

To address occlusion issues in detection, another study introduced a particle filter-based algorithm that estimates camera pose for vision-based tasks. It overcame traditional limitations like uncertainty and noise by integrating particle filtering for improved accuracy and robustness in real-time environments [8]. Similarly, another system proposed a smartphone-compatible sign-language recognition model using Mediapipe for hand landmark detection and particle filtering for smooth, frame-to-frame tracking, enabling accurate gesture recognition [7].

For posture correction, one study developed a webcam-based system using OpenCV and Mediapipe to calculate joint

angles and detect body landmarks. They verified accuracy in push-up repetitions, achieving success rates of 100%, 100%, 67%, 75%, and 80%, and provided real-time visual and audio feedback [9]. Another system, “Fitness Done Right,” used two two-branch multi-stage CNNs to detect movements, recognize exercises, and provide corrective feedback, reducing the error rate to 1.2% [10]. For exercise recognition, another study proposed an efficient HAR system combining TCN and Inception modules, capturing complex temporal patterns and achieving high accuracy across several datasets. The model was later compared with other deep learning methods and showed strong performance [11]. Finally, MotionMixer, an MLP-based model, was introduced for forecasting 3D human body motion. Using spatial MLPs to capture joint correlations and temporal MLPs for motion dynamics, it achieved slightly better results than baseline models when tested on Human3.6M, AMASS, and 3DPW datasets [12].

III. PROPOSED METHODOLOGY

A. Dataset Preparation

This research is based on video data of workouts, so we needed to collect a video dataset and then annotate the videos of the dataset so that we can have the right landmark positions for use as ground truth.

1) *Dataset Collection*: For this research, we collected publicly available videos from Kaggle datasets, YouTube, and official sites. For videos sourced from YouTube and official sites such as DareBee, Opex, and NASM. For videos that we obtained from YouTube and official sources, we contacted the respective content creators and secured permission for research use. To further enhance the dataset, we included our own videos under controlled conditions.

2) *Dataset Annotation*: After collecting the dataset, the next step was annotation. To do so, MediaPipe combined with a particle filter pipeline was used to annotate the dataset.

3) *Dataset Normalization*: To make our models robust across different body types and camera distances, we applied hip-centering normalization. Since torso proportions vary significantly between individuals, directly using raw coordinates can make posture detection and correction challenging. We calculate the hip center and store the relative x, y, z coordinates of 12 key landmarks (LEFT SHOULDER, RIGHT SHOULDER, LEFT ELBOW, RIGHT ELBOW, LEFT WRIST, RIGHT WRIST, LEFT HIP, RIGHT HIP, LEFT KNEE, RIGHT KNEE, LEFT ANKLE, RIGHT ANKLE).

Given the 3D landmarks from MediaPipe

$$L_t^{(k)} = [x_t^{(k)}, y_t^{(k)}, z_t^{(k)}] \quad (1)$$

where k indexes the landmark at frame t , the hip center is computed as

$$h_t = \frac{1}{2} (l_t^{(\text{left_hip})} + l_t^{(\text{right_hip})}). \quad (2)$$

Each landmark is then translated into a hip-centered coordinate system:

$$\tilde{l}_t^{(k)} = l_t^{(k)} - h_t. \quad (3)$$

4) *Dataset Augmentation*: To ensure the pose correction model performs effectively on real-world data, a data augmentation strategy is used during training. The model was trained not on the raw annotated landmarks data alone, but on a synthetically generated dataset designed to teach it how to handle noisy inputs. For each frame, add a small amount of Gaussian noise, and sometimes it hides the landmarks. The technique effectively trains the model as a denoising autoencoder.

5) *Feature Engineering*: The raw mediapipe detection was used, and then the angle was calculated to feed the model. For this, 8 major angles were selected (LEFT ELBOW ANGLE, RIGHT ELBOW ANGLE, LEFT SHOULDER ANGLE, RIGHT SHOULDER ANGLE, LEFT HIP ANGLE, RIGHT HIP ANGLE, LEFT KNEE ANGLE, RIGHT KNEE ANGLE). This helps the model to understand the posture more accurately.

B. Pose Landmark Estimation using MediaPipe

MediaPipe Pose is a framework developed by Google that can detect 33 key landmarks across the entire human body. In this project, it was used to detect joint points with a visibility confidence threshold of 0.5. Each video frame, captured using OpenCV, was processed through the MediaPipe model to extract pose landmarks. Lines were drawn between detected points to visualize the skeletal structure in real time.

C. Adding a Filter for Smooth Tracking

To improve the temporal consistency of the detected landmarks and mitigate issues like jitter or temporary occlusions, this paper used Particle Filtering. It only activates when the detection confidence of MediaPipe is less than 0.6, and the rest of the time, detection is continued using MediaPipe's detection.

1) *Adding a Filter for smooth tracking*: To improve the temporal consistency of the detected landmarks and mitigate issues like jitter or temporary occlusions, this paper used Particle Filtering. It only activates when the detection confidence of MediaPipe is less than 0.6, and the rest of the time, detection is continued using MediaPipe's detection.

2) *Particle Filtering*: A Particle Filter was implemented for each landmark to estimate its position.

- **Particles**: A set of NUM PARTICLES (1000) was used to represent the probability distribution of the landmark's state.
- **State**: Each particle represented the state as a 3D vector $[p_x, p_y, p_z]$. Let the state of a landmark at time t be defined as

$$\mathbf{x}_t = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}, \quad \mathbf{z}_t = \begin{bmatrix} \hat{p}_x \\ \hat{p}_y \\ \hat{p}_z \end{bmatrix} \quad (4)$$

where $\mathbf{z}_t$ is the observation from MediaPipe. The filter maintains N particles

$$\{\mathbf{x}_t^{(i)}, w_t^{(i)}\}_{i=1}^N \quad (5)$$

where $w_t^{(i)}$ denotes the normalized weight.

Process:

- Initialization: For each landmark, the particles were initialized around the first high-confidence MediaPipe measurement, and weights were set uniformly.

$$\mathbf{x}_0^{(i)} \sim \mathcal{N}(\mathbf{z}_0, \Sigma_0), \quad w_0^{(i)} = \frac{1}{N} \quad (6)$$

The particles are sampled around the first high-confidence measurement $\mathbf{z}_0$.

- Prediction Step: All particles were propagated according to the defined process model.

$$\mathbf{x}_{t-1}^{(i)} = f(\mathbf{x}_{t-1}^{(i)}) + \boldsymbol{\eta}_t^{(i)}, \quad \boldsymbol{\eta}_t^{(i)} \sim \mathcal{N}(0, \Sigma_p) \quad (7)$$

where $f(\cdot)$ denotes the process model (which can be the identity function if no explicit dynamics are assumed).

- Update Step: Particle weights were updated using the latest MediaPipe measurements and their associated confidence scores.

$$w_t^{(i)} \propto w_{t-1}^{(i)} \cdot p(\mathbf{z}_t | \mathbf{x}_t^{(i)}) \quad (8)$$

with the Gaussian likelihood function:

$$p(\mathbf{z}_t | \mathbf{x}_t^{(i)}) = c_t \cdot \exp\left(-\frac{1}{2} \left(\mathbf{z}_t - \mathbf{x}_t^{(i)}\right)^T \Sigma_m^{-1} \left(\mathbf{z}_t - \mathbf{x}_t^{(i)}\right)\right) \quad (9)$$

where c_t is the MediaPipe's visibility confidence.

- Normalization: After calculating the weight, it normalizes the value.

$$w_t^{(i)} = \frac{w_t^{(i)}}{\sum_{j=1}^N w_t^{(j)}} \quad (10)$$

- Estimation: The state of each landmark was estimated as the weighted average of all particles.

$$\hat{\mathbf{x}}_t = \sum_{i=1}^N w_t^{(i)} \mathbf{x}_t^{(i)} \quad (11)$$

D. Classification using TCN

This research used TCN (Temporal Convolutional Network) to classify exercises like push-ups and squats. The model analyzes short sequences (20 frames) of joints angles of 3D landmarks derived from Mediapipe, instead of using raw video.

$$h_t^{(l)} = \sigma\left(\sum_{k=0}^{K-1} W_k^{(l)} \cdot h_{t-d \cdot k}^{(l-1)} + b^{(l)}\right) \quad (12)$$

where

- $h_t^{(l)}$ is the hidden representation at layer l ,
- $W_k^{(l)}$ are convolutional filters,
- d is the dilation factor,
- K is the kernel size,
- $\sigma(\cdot)$ is a non-linear activation function (ReLU).

We also applied data augmentation and handled class imbalance with class weights to make the model more reliable.

The final network includes a TCN layer, dropout to reduce overfitting, and a softmax layer for class prediction.

The model was trained with the Adam optimizer and early stopping, and it performed well in distinguishing between push-ups, squats, and no activity. Finally, we converted the model to TensorFlowLite, so it can run efficiently on low-power devices.

E. Correction using MLP

To complement our classifier, we developed pose correction models for push-ups and squats using an MLP (Multi-Layer Perceptron). 2 MLP models issued to use push-ups and squats. These models learn to correct imperfect form by training on ideal 3D body landmark data that have been slightly distorted with random noise. By learning to "denoise" these data, the models can take the user's real-time movements and predict the ideal posture, providing live feedback and guidance. The model was trained in correctly performed exercises of different lengths, which allows it to learn the expected posture at each stage of the movement. Based on this, the model can predict the ideal biomechanical posture for each frame and shows it as a "Ghost Line" overlay. Whether the user is doing exercise correctly or in the wrong form, it will help the user to track whether the user is on the correct form or not.

The denoising network is an MLP (Multi-Layer Perceptron) defined as:

$$\mathbf{h}_1 = \phi(W_1 \tilde{\mathbf{x}}_t + b_1), \quad \mathbf{h}_2 = \phi(W_2 \mathbf{h}_1 + b_2), \quad \mathbf{h}_3 = \phi(W_3 \mathbf{h}_2 + b_3) \quad (13)$$

with ReLU activation $\phi(\cdot)$, *batch normalization*, and *dropout* applied after intermediate layers to improve generalization.

The output layer uses a linear activation to predict corrected landmark coordinates:

$$\hat{\mathbf{y}}_t = W_0 \mathbf{h}_3 + b_0 \quad (14)$$

These models are built using a Multi-Layer Perceptron (MLP) with several dense layers, batch normalization, and dropout to prevent overfitting.

F. Proposed Workflow

The workflow starts at data collection, and we divide the data into training (70%), validation (15%) and test (15%). We pre-processed that data by annotating them using MediaPipe and a particle filter pipeline and used hip centering for normalizing and augmentation by adding noise to ensure generalization of the model. After that joint angles are calculated from the detected landmarks, which serve as features for training the Temporal Convolutional Network (TCN) to recognize exercises. Once preprocessing is complete, both models are trained and later evaluated on the test set. Finally, the workflow outputs exercise classification and correction, ensuring consistency across training, validation, and testing phases. The workflow of the system is shown in Fig 1.

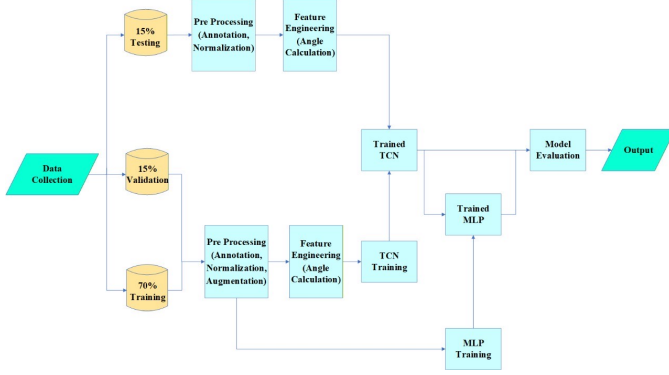


Fig. 1. Workflow of the system

G. Privacy & Implementation

This framework uses a data minimization technique to protect user privacy. Raw video is instantly transformed into anonymous skeletal landmarks by using MediaPipe. No identifiable video footage is stored, examined, or sent to external servers because the models (TCN and MLP) only work with these landmark coordinate vectors.

IV. EXPERIMENTAL RESULT AND ANALYSIS

A. Experimental Setup

All experiments were carried out on a desktop with an Intel Pentium Core 2 Duo processor, 16 GB of RAM, and an AMD Radeon RX 580 GPU. The environment was set up using Visual Studio Code on Windows 10 (64-bit), with Python 3.10.11. MediaPipe (v0.10.9) was used for the detection model, and TensorFlow (v2.19.1) was used to implement the particle filtering, MLP, and TCN. Our experimental setup prioritized performance efficiency while balancing computational constraints. The architecture is to operate stage by stage; first it detects landmarks using MediaPipe and if occlusion occurs, it uses the Particle filter; otherwise, the MediaPipe output is used. By processing the landmark vectors (extracted via BlazePose) instead of raw video, the computational cost is reduced. TCN processes the landmarks input across 20 frames to classify, and MLP is triggered to do the forecast. Furthermore, TFLite allows the system to execute on low-spec devices also.

B. Results

1) *Filter Evaluation:* The performance of the raw MediaPipe detections and the filtering approach was quantitatively evaluated against the ground truth data using MPJPE, PCK, and AUC for PCK. Table I demonstrates that the Particle filtering method outperforms raw MediaPipe across all metrics, achieving better MPJPE, PCK, and AUC.

TABLE I
COMPARISON OF LANDMARK ESTIMATION METHODS ACROSS MPJPE, PCK, AND AUC METRICS.

Metric	Raw Mediapipe	Particle Filtering	Best Performer
MPJPE	36.9067 pixels	32.9584 pixels	Particle Filtering
PCK	98.83%	99.17%	Particle Filtering
AUC (PCK Curve)	0.9094	0.9265	Particle Filtering

2) Classifier:

a) *Classification Report:* The classification report summarizes the model's performance on the test set. Overall accuracy is 98%, with per-class F1-scores of 0.99 (PUSHUP), 0.98 (SQUAT), and 0.95 (NONE). Table II demonstrates that the model can correctly identify each exercise while effectively distinguishing it from other movements.

TABLE II
CLASSIFICATION REPORT SHOWING PRECISION, RECALL, F1-SCORE, AND SUPPORT FOR EACH CLASS.

Class	Precision	Recall	F1-score	Support
PUSHUP	0.99	0.99	0.99	489
SQUAT	0.98	0.98	0.98	753
NONE	0.96	0.94	0.95	364
Accuracy			0.98	1606
Macro Avg	0.97	0.97	0.97	1606
Weighted Avg	0.98	0.98	0.98	1606

b) *Validation vs Test Accuracy:* The model accuracy over epochs is shown in Fig 2.

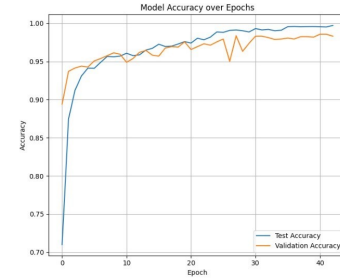


Fig. 2. Validation vs Test Accuracy

c) *Validation vs Test Loss:* The model loss over epochs is shown in Fig 3.

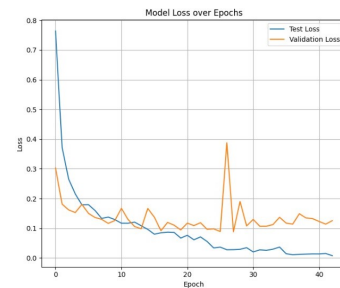


Fig. 3. Validation vs Test Loss

d) *Precision Recall Curve:* The precision-recall curves further confirm the model's reliability, especially for imbalanced data. All curves are near the top-right corner, indicating high precision and recall. The Average Precision (AP) scores are 1.00 for "PUSHUP" and "SQUAT" and 0.99 for "NONE," highlighting consistently excellent performance. The multi-class precision-recall curve is shown in Fig 4.

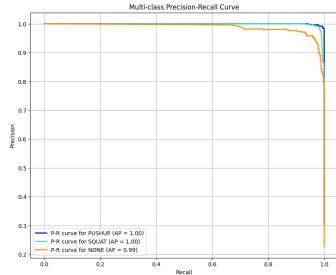


Fig. 4. Precision Recall Curve

e) *F1 Score Curve:* This analysis shows the F1-score for each class across decision thresholds from 0.0 to 1.0. "PUSHUP" and "SQUAT" maintain near-perfect F1-scores (0.99) at high thresholds (0.72 and 0.82). This confirms that setting a high confidence threshold in the real-time system ensures reliable predictions. The F1 score vs. decision threshold is shown in Fig 5.

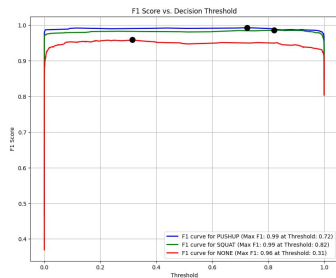


Fig. 5. F1 Score Curve

f) *Confusion Matrix:* The confusion matrix gives a clear picture of how well the classifier performed on the test set. The strong diagonal values (486, 741, and 341) show that the model correctly identified most samples of "PUSHUP," "SQUAT," and "NONE." Off-diagonal values are low, meaning there was very little confusion between classes. The main misclassifications occurred when some "NONE" samples were mistaken for "SQUAT" (17 instances). Overall, this matrix highlights the model's strong accuracy and ability to distinguish between different movements. The confusion matrix is shown in Fig 6.

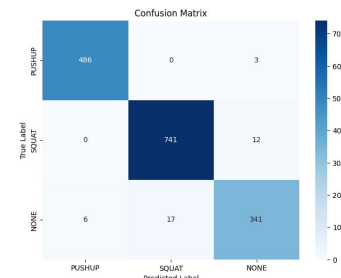


Fig. 6. Confusion Matrix

g) *ROC Curve:* The ROC curves for "PUSHUP," "SQUAT," and "NONE" are all positioned near the top-left corner, reflecting high true positive rates and low false positive rates. Each class achieved an AUC of 1.00, demonstrating near-perfect classification performance across all thresholds. The multi class ROC curve is shown in Fig 7.

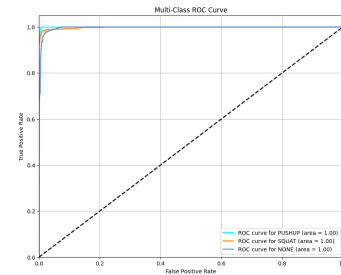


Fig. 7. ROC Curve

h) *Classifier Test Accuracy:*
The Classifier Test accuracy is : 97.63%.

3) Regression:

a) Mean Squared Error:

- For Pushup : 0.072°
- For Squat : 0.09°

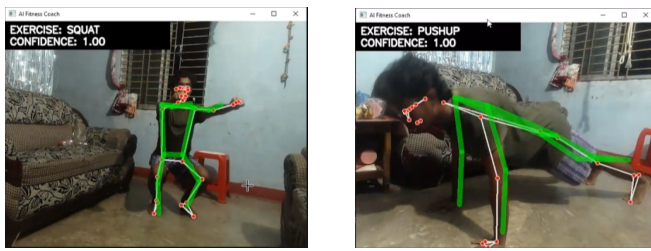
b) Mean Absolute Error:

- For Pushup : 2.68°
- For Squat : 2.86°

C. Output

Qualitative testing showed significant benefits for beginners. For example, during push-ups, the system effectively corrected common mistakes such as keeping the arms too wide by projecting a ghost line that guides the elbows backward. This way the system shows its potential to prevent injuries caused by repeatedly performing exercises with improper form.

The outputs are visually shown in Fig 8. where the first image displays a squat, and the system successfully classifies that with a confidence of 1. The second image shows a push-up, and the system successfully classifies that with a confidence of 1. Both figures illustrate the live output of exercise detection and the result of the system.



(a) Squat (b) Pushup

Fig. 8. Squat and Pushup Live

V. CONCLUSION & FUTURE WORK

In recent years, many smart fitness products—like workout mirrors, treadmills, and weightlifting devices—have appeared on the market. However, these technologies are often too expensive for most people. While gym memberships can be affordable, hiring a personal trainer to ensure correct exercise form adds significant cost. Although some devices collect daily fitness data, they rarely provide guidance on proper exercises. In contrast, most people already own a mobile phone or laptop equipped with a camera and display. Our research proposes a cost-effective system that uses these accessible devices to detect posture through deep learning and provide real-time visual feedback. This helps users avoid injuries and exercise safely anytime, anywhere, without needing costly equipment or a professional trainer.

In its current stage, the system recognizes only two exercises—push-ups and squats—which limits its overall coverage. It also depends on environmental conditions; low light or multiple people in the frame can reduce accuracy since the model relies on 3D body coordinates. Additionally, because feedback is visual, users must keep checking the screen during their workout, which can be distracting.

Despite these constraints, the research shows strong potential. Future development could include training the model with larger datasets to support more exercises, enabling full-body workout monitoring. A major improvement would be integrating audio feedback, allowing the system to give real-time verbal cues like “Straighten your back” or “Nice squat!” This would make the experience more intuitive, hands-free, and user-friendly, even when the screen isn’t visible.

REFERENCES

- [1] D. A. Bonilla *et al.*, “Exercise Selection and Common Injuries in Fitness Centers: A Systematic Integrative Review and Practical Recommendations,” *International Journal of Environmental Research and Public Health*, vol. 19, no. 19, Oct. 2022. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/36232010/>
- [2] M. Foley, “The 5 Best Fitness Mirrors of 2025, Tested & Approved,” *Verywell Fit*, Feb. 19, 2025. [Online]. Available: <https://www.verywellfit.com/best-fitness-mirror-5208223>
- [3] J. K. Ngo, P. Solis-Urra, and J. Sanchez-Martinez, “Injury Profile Among Street Workout Practitioners,” *Orthopaedic Journal of Sports Medicine*, vol. 9, no. 6, 2021. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8209839/>

- [4] R. O. Ogundokun, R. Maskeliūnas, and R. Damaševičius, “Human Posture Detection Using Image Augmentation and Hyperparameter-Optimized Transfer Learning Algorithms,” *Applied Sciences*, vol. 12, no. 19, Art. ID 10156, Oct. 2022. [Online]. Available: <https://www.mdpi.com/2076-3417/12/19/10156>
- [5] K. S. Sim, S. Wong, A. Low, A. P. Yunus, and C. Lim, “Real-Time Digital Assistance for Exercise: Exercise Tracking System with MediaPipe Angle Directive Rules,” *International Journal on Informatics Visualization (JOIV)*, vol. 8, no. 4, pp. 2380–2387, Dec. 2024. [Online]. Available: <https://doi.org/10.62527/joiv.8.4.2993>
- [6] S. D. Kale, N. Kulkarni, S. Kumbhkar, and S. Kharde, “Posture detection and comparison of different physical exercises based on deep learning using media pipe,opencv,” ResearchGate, Apr. 2023, [Online]. Available: <https://www.researchgate.net/publication/369927493>
- [7] A. H. A. Hafez and E. Cervera, “Particle-filter-based Pose Estimation from Controlled Motion with Application to Visual Servoing,” *International Journal of Advanced Robotic Systems*, vol. 11, no. 10, Oct. 2014, <https://doi.org/10.5772/58928>.
- [8] H. M. A. Ghanimi *et al.*, “An open-source MP + CNN + BiLSTM model-based hybrid model for recognizing sign language on smartphones,” *International Journal of Systems Assurance Engineering and Management*, vol. 15, no. 8, pp. 3794–3806, May 2024, <https://doi.org/10.1007/s13198-024-02376-x>.
- [9] Y. Kwon and D. Kim, “Real-Time Workout Posture Correction using OpenCV and MediaPipe,” *The Journal of Korean Institute of Information Technology*, vol. 20, no. 1, pp. 199–208, Jan. 2022, <https://doi.org/10.14801/jkiit.2022.20.1.199>.
- [10] Y. Chen, Y. Chen, and Z. Tu, “Fitness Done Right: a Real-time Intelligent Personal Trainer for Exercise Correction,” *arXiv.org*, Oct. 30, 2019. <https://arxiv.org/abs/1911.07935>
- [11] M. A. A. Al-Qaness, A. Dahou, N. T. Trouba, M. A. Elaziz, and A. M. Helmi, “TCN-Inception: Temporal Convolutional Network and Inception modules for sensor-based Human Activity Recognition,” *Future Generation Computer Systems*, vol. 160, pp. 375–388, Jun. 2024, <https://doi.org/10.1016/j.future.2024.06.016>.
- [12] A. Bouazizi, A. Holzbock, U. Kressel, K. Dietmayer, and V. Belagiannis, “MotionMixer: MLP-based 3D Human body pose Forecasting,” *arXiv (Cornell University)*, Jan. 2022, <https://doi.org/10.48550/arxiv.2207.00499>.